

I. THE ROLE OF SQL IN THE BIG DATA ERA

While NoSQL and Unstructured data are popular, **SQL** remains the universal language of data. In modern Big Data stacks (like Snowflake, Google BigQuery, or Amazon Redshift), SQL is used to transform raw data into "Gold" tables for Machine Learning models. For a Data Engineer, writing a query is easy; writing a **performant** query on 100 Terabytes of data is the real challenge.

1.1. From SQL to Big Data SQL

Traditional SQL (MySQL/PostgreSQL) runs on a single machine. Big Data SQL engines (Hive, Presto, SparkSQL) use **Distributed Query Execution**, where a single query is broken into small tasks and run across hundreds of servers simultaneously.

II. ADVANCED JOIN STRATEGIES & OPTIMIZATION

In Big Data, joining two massive tables can cause "Data Skew" or "Out of Memory" errors. Understanding how the engine joins data is critical.

2.1. Common Join Types Recap

- **Inner, Left, Right, Full Outer:** (Refer to File 7 for basics).
- **Self-Join:** Joining a table with itself (e.g., finding employees who earn more than their managers in a staff table).

2.2. Big Data Join Mechanics

- **Broadcast Join:** If one table is small (e.g., a "City" lookup table), the engine sends a copy of it to every server. This is extremely fast.
- **Shuffle Hash Join:** When both tables are large, the engine re-distributes data across the network based on the Join Key. This is expensive in terms of time and network bandwidth.

III. WINDOW FUNCTIONS: THE POWER ANALYTICS TOOL

Window Functions allow you to perform calculations across a set of rows related to the current row without collapsing them into a single output (unlike GROUP BY).

3.1. Syntax Structure

SELECT column, FUNCTION() OVER (PARTITION BY group_col ORDER BY sort_col) FROM table

3.2. Essential Window Functions

- **ROW_NUMBER():** Assigning a unique ID to rows within a group (e.g., finding the latest login for every user).
- **RANK() & DENSE_RANK():** Used for competitive scoring (e.g., ranking students by GPA per major).
- **LAG() & LEAD():** Accessing data from the previous or next row. This is vital for **Time-Series Analysis** (e.g., calculating the percentage growth in sales compared to yesterday).
- **SUM() OVER / AVG() OVER:** Calculating running totals or moving averages.

IV. COMMON TABLE EXPRESSIONS (CTE) & RECURSION

Writing massive, nested subqueries makes code unreadable and hard to debug. **CTEs** (WITH clause) allow you to define temporary result sets that exist only during the execution of the query.

4.1. Benefits of CTEs

1. **Readability:** Breaking a 200-line query into small, named logic blocks.
2. **Reusability:** Referencing the same logic block multiple times in one query.
3. **Recursion:** Using WITH RECURSIVE to query hierarchical data, such as organizational charts or folder structures.

V. QUERY PERFORMANCE TUNING FOR BIG DATA

In a cloud environment (AWS/GCP), an unoptimized query doesn't just run slowly—it costs more money.

5.1. Filtering and Pruning

- **Partition Pruning:** Only scanning specific folders/partitions (e.g., WHERE year = 2026). Scanning only 1% of the data saves 99% of the cost.
- **Predicate Pushdown:** Filtering data as early as possible in the execution plan to reduce the amount of data moving through the pipeline.

5.2. Columnar Storage vs. Row Storage

Big Data engines use formats like **Parquet** or **ORC**. Unlike CSV (row-based), these store data by **Column**.

- **Engine Benefit:** If you only SELECT price, the engine skips reading the name, address, and id columns entirely.

VI. PRACTICAL LAB: COMPLEX DATA TRANSFORMATION

SQL

-- Goal: Find the top 3 highest spending customers per country

-- who spent more than the average of their country.

WITH CustomerSpending AS (

SELECT

customer_id,

country,

```

SUM(total_amount) as total_spent

FROM orders

GROUP BY customer_id, country

),

CountryAverage AS (

SELECT

country,

AVG(total_spent) as avg_spent

FROM CustomerSpending

GROUP BY country

)

SELECT * FROM (

SELECT

cs.customer_id,

cs.country,

cs.total_spent,

RANK() OVER (PARTITION BY cs.country ORDER BY cs.total_spent DESC) as spending_rank

FROM CustomerSpending cs

JOIN CountryAverage ca ON cs.country = ca.country

WHERE cs.total_spent > ca.avg_spent

) ranked_data

WHERE spending_rank <= 3;

```